

Exercise 3: Case statements

Nolwandle Lala Dhladhla

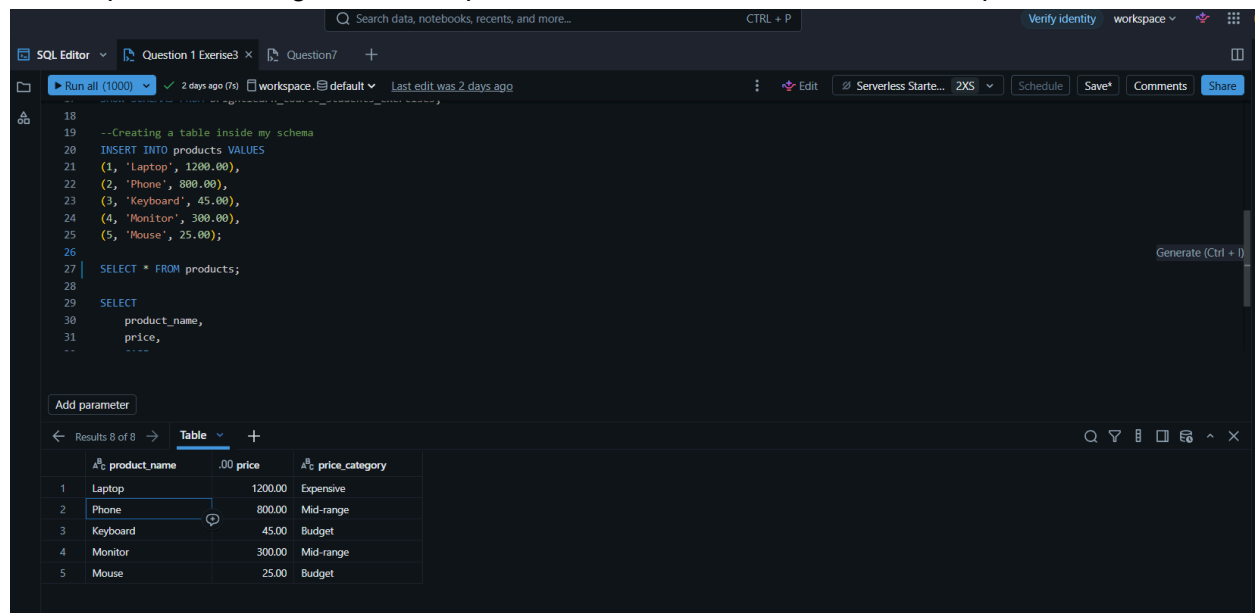
Data Analytics

Bright learn

Date : 11 September 2026

1: The products Table

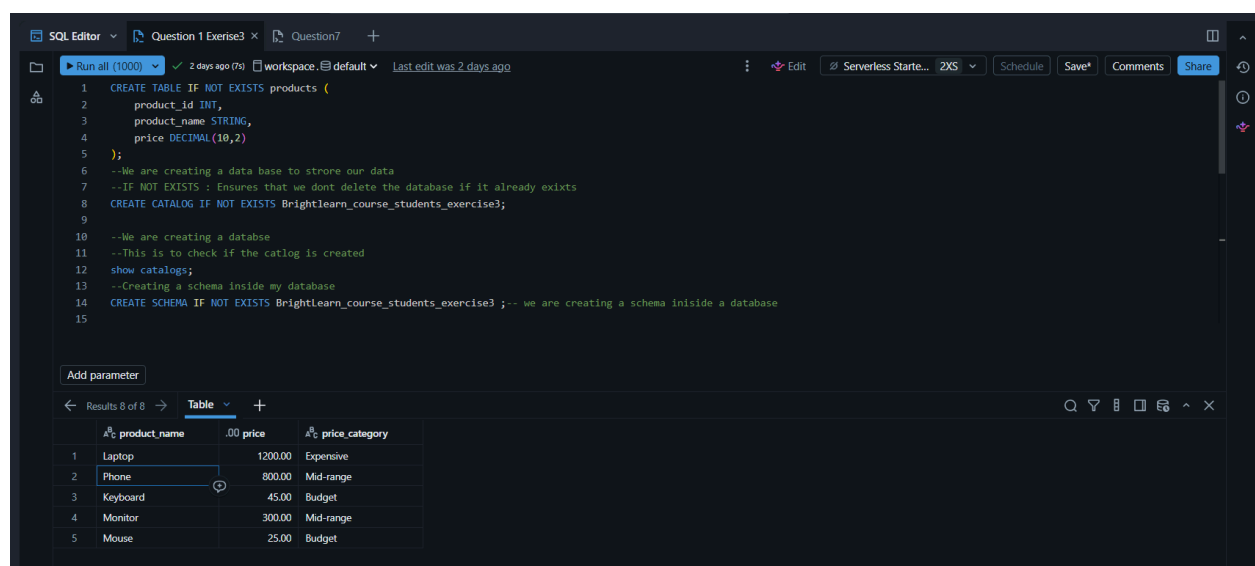
A short product catalogue. Use the price column to bucket each item into a price tier



The screenshot shows a SQL Editor interface with a dark theme. The top bar includes a search bar, a workspace dropdown, and a 'Verify identity' button. The main editor area contains SQL code for creating a table and inserting data. Below the code, the results of the query are displayed in a table format.

```
18
19 --Creating a table inside my schema
20 INSERT INTO products VALUES
21 (1, 'Laptop', 1200.00),
22 (2, 'Phone', 800.00),
23 (3, 'Keyboard', 45.00),
24 (4, 'Monitor', 300.00),
25 (5, 'Mouse', 25.00);
26
27 SELECT * FROM products;
28
29 SELECT
30     product_name,
31     price,
```

product_name	price	price_category
Laptop	1200.00	Expensive
Phone	800.00	Mid-range
Keyboard	45.00	Budget
Monitor	300.00	Mid-range
Mouse	25.00	Budget



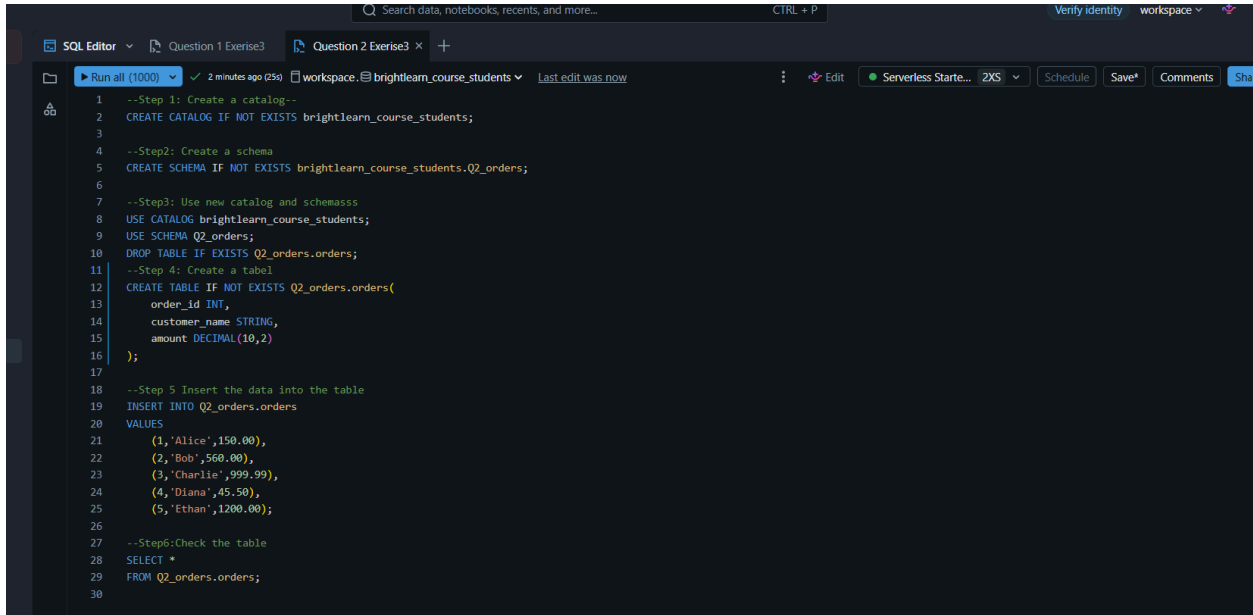
The screenshot shows a SQL Editor interface with a dark theme. The top bar includes a search bar, a workspace dropdown, and a 'Verify identity' button. The main editor area contains SQL code for creating a table and inserting data. Below the code, the results of the query are displayed in a table format.

```
1 CREATE TABLE IF NOT EXISTS products (
2     product_id INT,
3     product_name STRING,
4     price DECIMAL(10,2)
5 );
6 --We are creating a data base to store our data
7 --IF NOT EXISTS : Ensures that we dont delete the database if it already exists
8 CREATE CATALOG IF NOT EXISTS Brightlearn_course_students_exercise3;
9
10 --We are creating a database
11 --This is to check if the catlog is created
12 show catalogs;
13 --Creating a schema inside my database
14 CREATE SCHEMA IF NOT EXISTS Brightlearn_course_students_exercise3 ;-- we are creating a schema inside a database
15
```

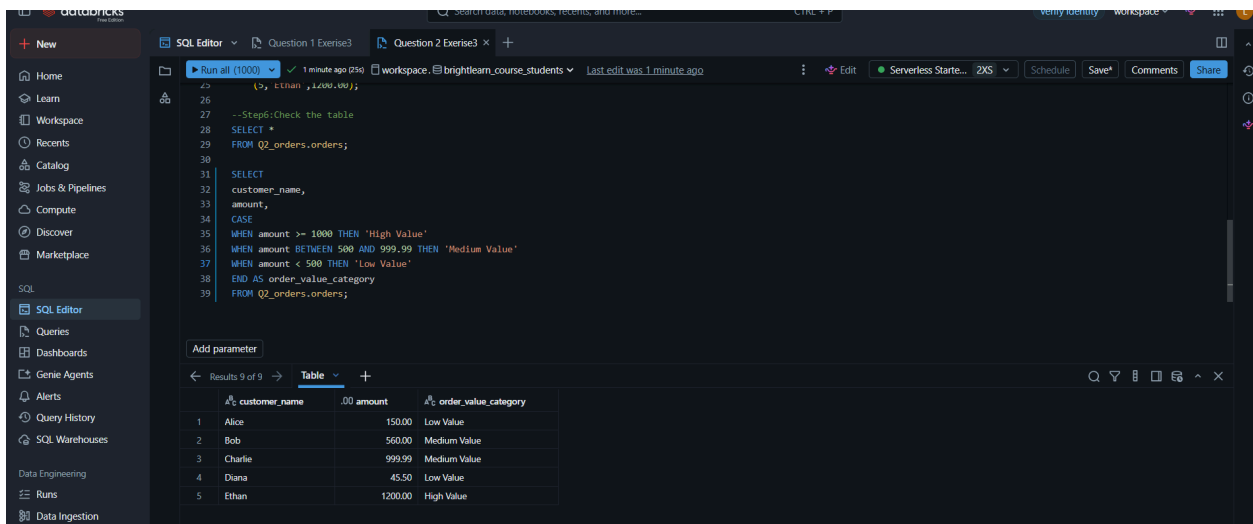
product_name	price	price_category
Laptop	1200.00	Expensive
Phone	800.00	Mid-range
Keyboard	45.00	Budget
Monitor	300.00	Mid-range
Mouse	25.00	Budget

2 : The Orders Table

Customer orders with the amount paid. Label each order by its monetary tier.



```
1  --Step 1: Create a catalog--
2  CREATE CATALOG IF NOT EXISTS brightlearn_course_students;
3
4  --Step2: Create a schema
5  CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.Q2_orders;
6
7  --Step3: Use new catalog and schemass
8  USE CATALOG brightlearn_course_students;
9  USE SCHEMA Q2_orders;
10 DROP TABLE IF EXISTS Q2_orders.orders;
11 --Step 4: Create a table
12 CREATE TABLE IF NOT EXISTS Q2_orders.orders(
13     order_id INT,
14     customer_name STRING,
15     amount DECIMAL(10,2)
16 );
17
18 --Step 5 Insert the data into the table
19 INSERT INTO Q2_orders.orders
20 VALUES
21 (1,'Alice',150.00),
22 (2,'Bob',560.00),
23 (3,'Charlie',999.99),
24 (4,'Diana',45.50),
25 (5,'Ethan',1200.00);
26
27 --Step6:Check the table
28 SELECT *
29 FROM Q2_orders.orders;
30
```



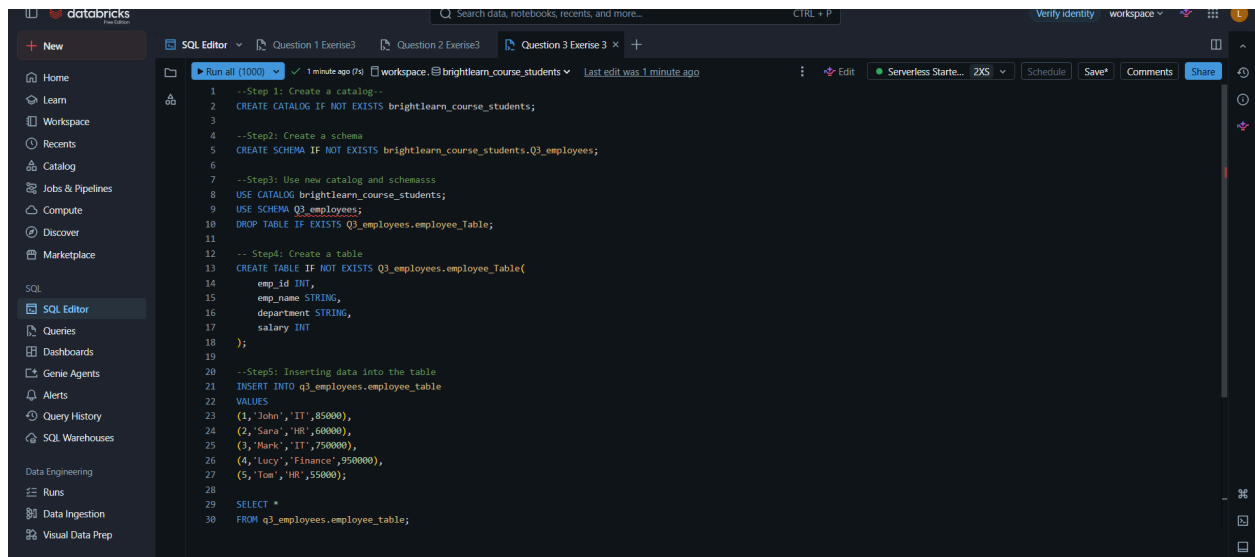
```
26 (5,'Ethan',1200.00);
27 --Step6:Check the table
28 SELECT *
29 FROM Q2_orders.orders;
30
31 SELECT
32     customer_name,
33     amount,
34     CASE
35     WHEN amount >= 1000 THEN 'High Value'
36     WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium Value'
37     WHEN amount < 500 THEN 'Low Value'
38     END AS order_value_category
39 FROM Q2_orders.orders;
```

Results 9 of 9

	customer_name	amount	order_value_category
1	Alice	150.00	Low Value
2	Bob	560.00	Medium Value
3	Charlie	999.99	Medium Value
4	Diana	45.50	Low Value
5	Ethan	1200.00	High Value

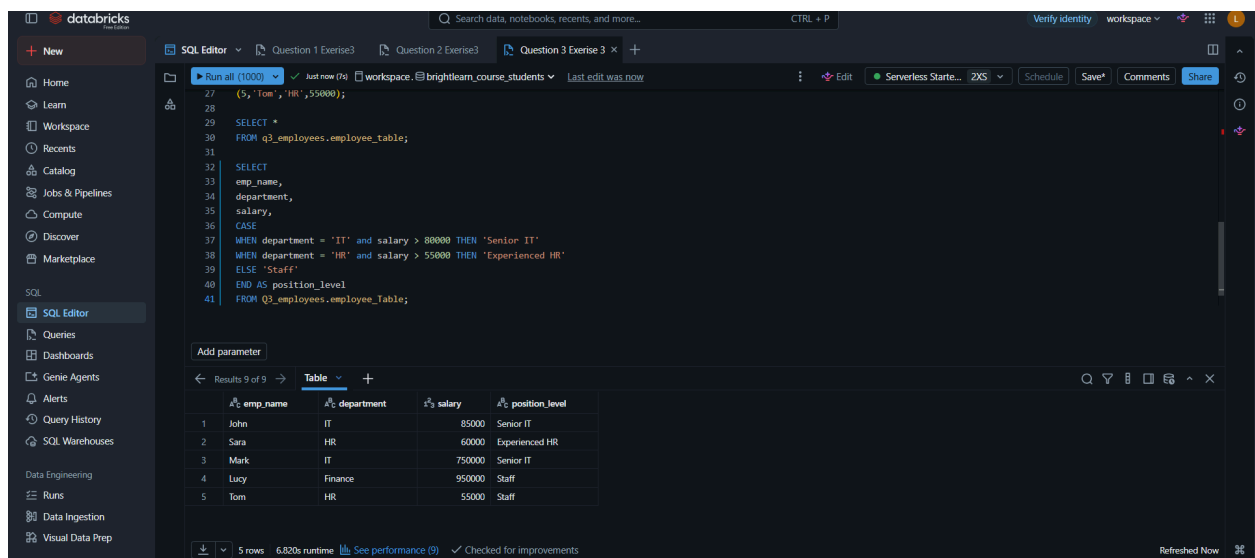
3 : The employees Table

Combine the department and salary columns to label each employee with a seniority tier



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area displays a SQL script with the following steps:

```
1 --Step 1: Create a catalog--
2 CREATE CATALOG IF NOT EXISTS brightlearn_course_students;
3
4 --Step2: Create a schema
5 CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.q3_employees;
6
7 --Step3: Use new catalog and schemass
8 USE CATALOG brightlearn_course_students;
9 USE SCHEMA q3_employees;
10 DROP TABLE IF EXISTS q3_employees.employee_table;
11
12 -- Step4: Create a table
13 CREATE TABLE IF NOT EXISTS q3_employees.employee_table(
14     emp_id INT,
15     emp_name STRING,
16     department STRING,
17     salary INT
18 );
19
20 --Step5: Inserting data into the table
21 INSERT INTO q3_employees.employee_table
22 VALUES
23 (1,'John','IT',85000),
24 (2,'Sara','HR',60000),
25 (3,'Mark','IT',750000),
26 (4,'Lucy','Finance',950000),
27 (5,'Tom','HR',55000);
28
29 SELECT *
30 FROM q3_employees.employee_table;
```

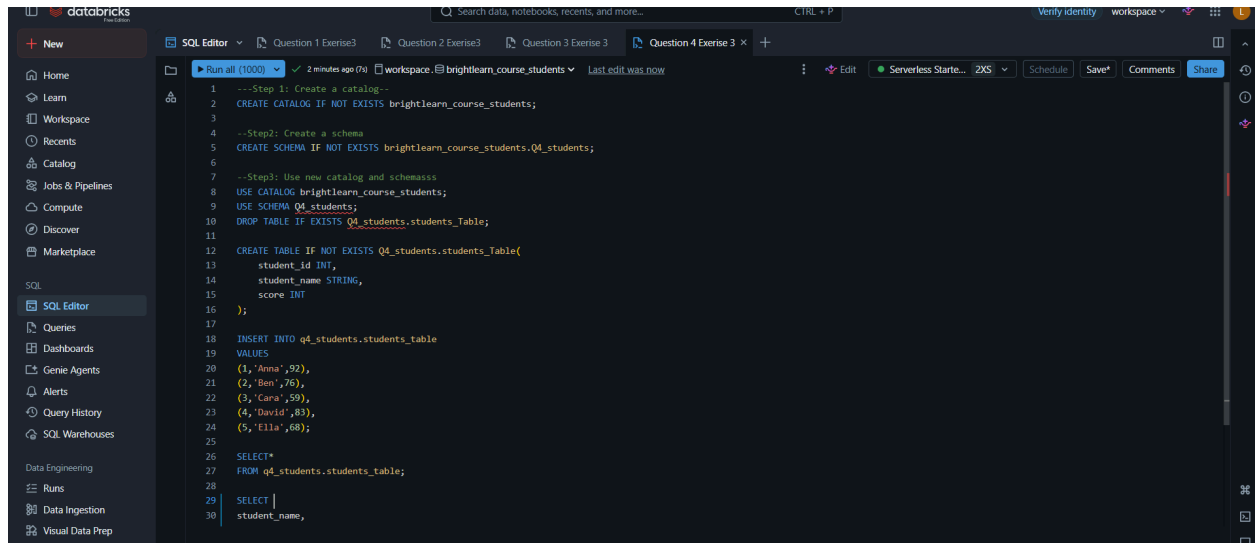


The screenshot shows the Databricks SQL Editor interface with the same SQL script as above. The results of the query are displayed in a table format below the script. The table has 5 rows and 4 columns: emp_name, department, salary, and position_level. The position_level is derived from the department and salary columns using a CASE statement.

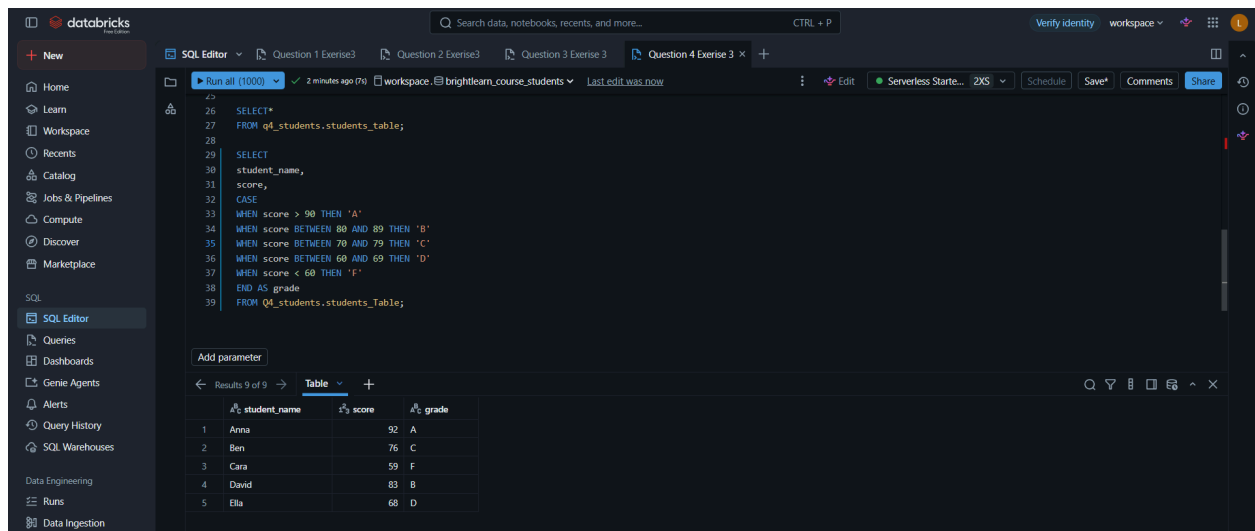
emp_name	department	salary	position_level
John	IT	85000	Senior IT
Sara	HR	60000	Experienced HR
Mark	IT	750000	Senior IT
Lucy	Finance	950000	Staff
Tom	HR	55000	Staff

4 : The students Table

A list of student scores. Convert numeric scores into letter grades.



```
1  --Step 1: Create a catalog--
2  CREATE CATALOG IF NOT EXISTS brightlearn_course_students;
3
4  --Step2: Create a schema
5  CREATE SCHEMA IF NOT EXISTS brightlearn_course_students.q4_students;
6
7  --Step3: Use new catalog and schemasss
8  USE CATALOG brightlearn_course_students;
9  USE SCHEMA q4_students;
10 DROP TABLE IF EXISTS q4_students.students_Table;
11
12 CREATE TABLE IF NOT EXISTS q4_students.students_Table(
13     student_id INT,
14     student_name STRING,
15     score INT
16 );
17
18 INSERT INTO q4_students.students_table
19 VALUES
20 (1,'Anna',92),
21 (2,'Ben',76),
22 (3,'Cara',59),
23 (4,'David',83),
24 (5,'Ella',68);
25
26 SELECT*
27 FROM q4_students.students_table;
28
29 SELECT |
30 student_name,
```

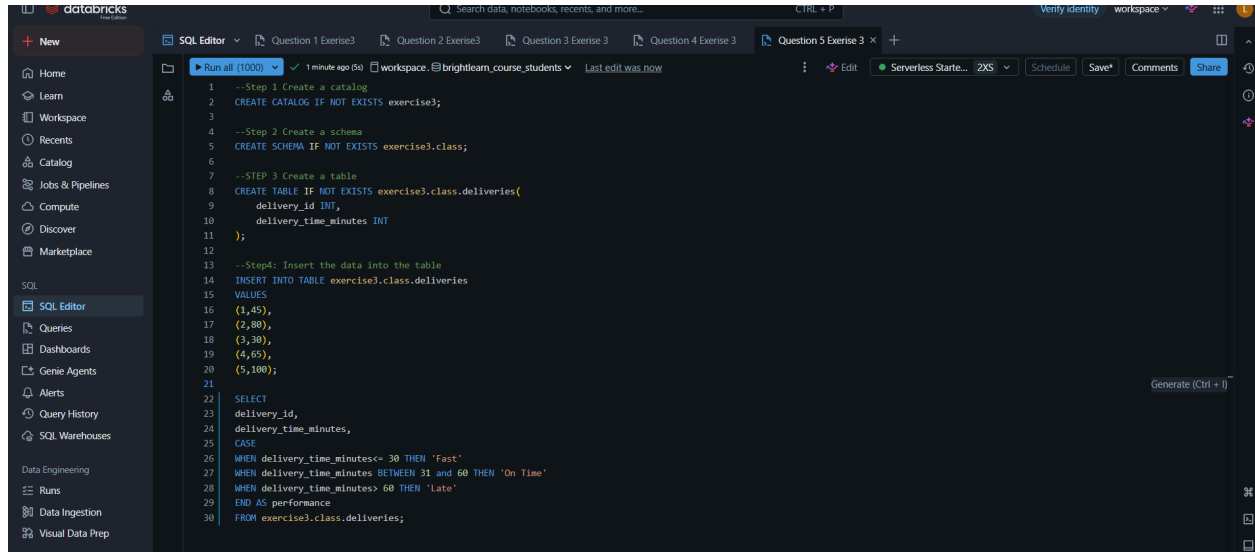


```
26 SELECT*
27 FROM q4_students.students_table;
28
29 SELECT
30     student_name,
31     score,
32     CASE
33     WHEN score > 90 THEN 'A'
34     WHEN score BETWEEN 80 AND 89 THEN 'B'
35     WHEN score BETWEEN 70 AND 79 THEN 'C'
36     WHEN score BETWEEN 60 AND 69 THEN 'D'
37     WHEN score < 60 THEN 'F'
38     END AS grade
39 FROM q4_students.students_Table;
```

student_name	score	grade
Anna	92	A
Ben	76	C
Cara	59	F
David	83	B
Ella	68	D

5. The deliveries Table

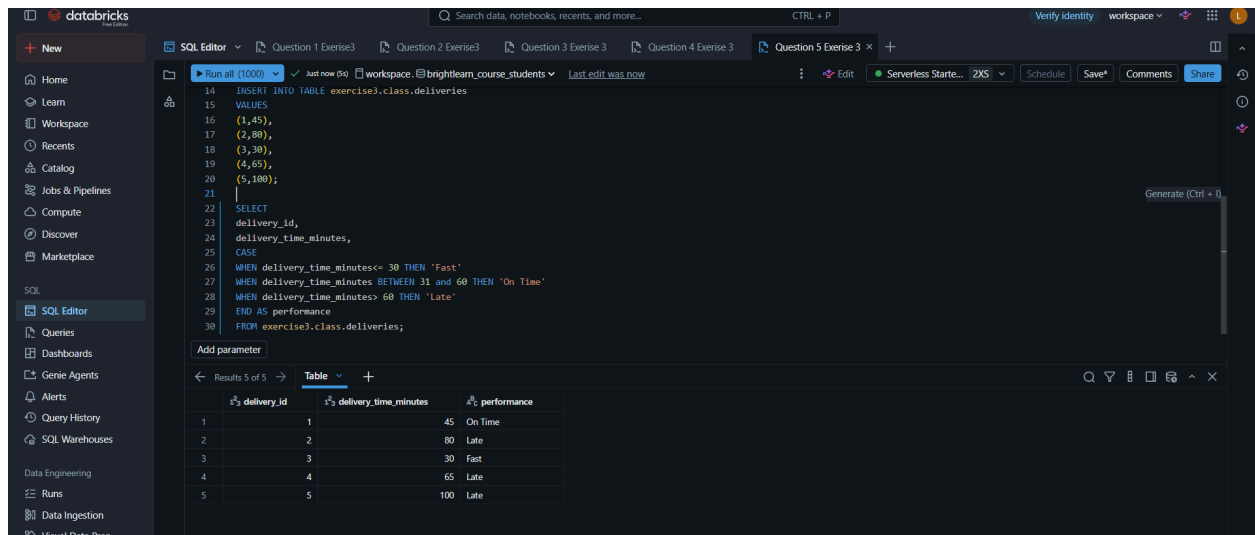
Delivery times in minutes. Label each delivery by its performance



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays a SQL script with the following content:

```
1 --Step 1 Create a catalog
2 CREATE CATALOG IF NOT EXISTS exercise3;
3
4 --Step 2 Create a schema
5 CREATE SCHEMA IF NOT EXISTS exercise3.class;
6
7 --Step 3 Create a table
8 CREATE TABLE IF NOT EXISTS exercise3.class.deliveries(
9   delivery_id INT,
10  delivery_time_minutes INT
11 );
12
13 --Step4: Insert the data into the table
14 INSERT INTO TABLE exercise3.class.deliveries
15 VALUES
16 (1,45),
17 (2,80),
18 (3,30),
19 (4,65),
20 (5,100);
21
22 SELECT
23   delivery_id,
24   delivery_time_minutes,
25   CASE
26     WHEN delivery_time_minutes <= 30 THEN 'Fast'
27     WHEN delivery_time_minutes BETWEEN 31 and 60 THEN 'On Time'
28     WHEN delivery_time_minutes > 60 THEN 'Late'
29   END AS performance
30 FROM exercise3.class.deliveries;
```

The script is executed, and the results are displayed in a table at the bottom of the editor.

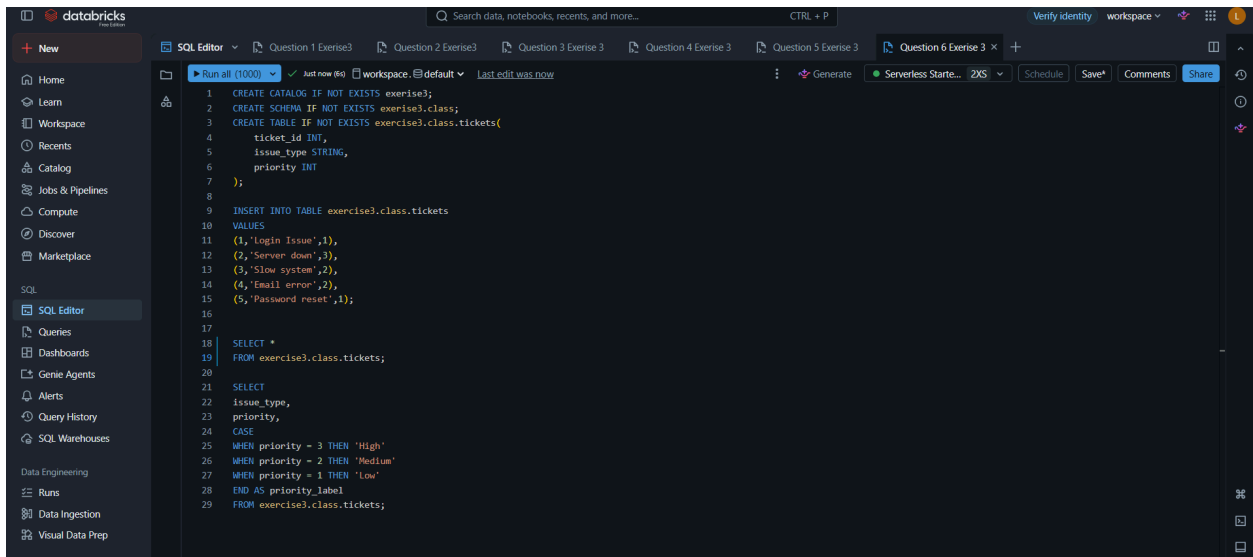


The screenshot shows the Databricks SQL Editor interface with the results of the SQL query displayed in a table. The table has 5 rows and 4 columns: `delivery_id`, `delivery_time_minutes`, and `performance`. The results are as follows:

<code>delivery_id</code>	<code>delivery_time_minutes</code>	<code>performance</code>
1	45	On Time
2	80	Late
3	30	Fast
4	65	Late
5	100	Late

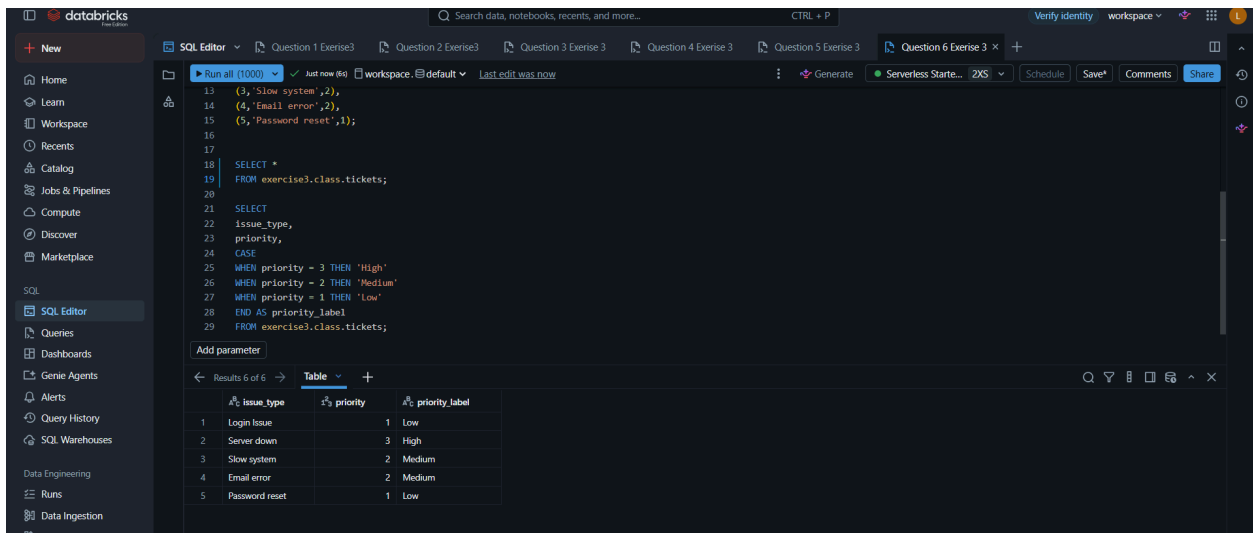
6. The Ticket Table

Support tickets stored with a numeric priority. Convert the priority into a readable label.



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays SQL code for creating a catalog, schema, and table, followed by an insert statement.

```
1 CREATE CATALOG IF NOT EXISTS exercise3;
2 CREATE SCHEMA IF NOT EXISTS exercise3.class;
3 CREATE TABLE IF NOT EXISTS exercise3.class.tickets(
4     ticket_id INT,
5     issue_type STRING,
6     priority INT
7 );
8
9 INSERT INTO TABLE exercise3.class.tickets
10 VALUES
11 (1,'Login Issue',1),
12 (2,'Server down',3),
13 (3,'Slow system',2),
14 (4,'Email error',2),
15 (5,'Password reset',1);
16
17
18 SELECT *
19 FROM exercise3.class.tickets;
20
21 SELECT
22     issue_type,
23     priority,
24     CASE
25     WHEN priority = 3 THEN 'High'
26     WHEN priority = 2 THEN 'Medium'
27     WHEN priority = 1 THEN 'Low'
28     END AS priority_label
29 FROM exercise3.class.tickets;
```

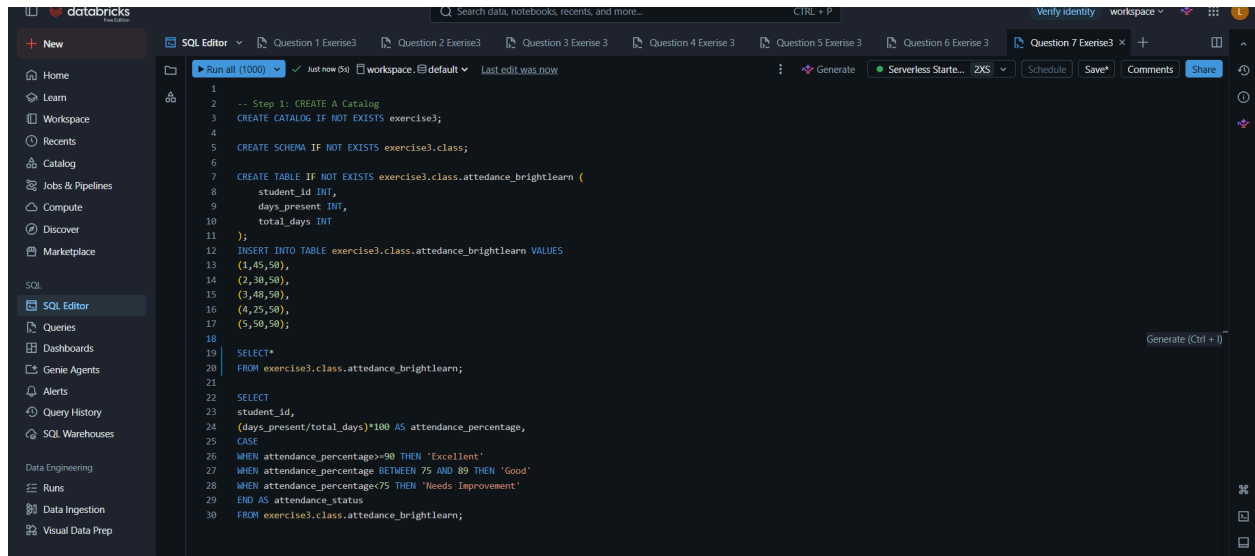


The screenshot shows the Databricks SQL Editor interface with the results of the SQL query displayed at the bottom. The results are shown in a table format with 6 columns: issue_type, priority, and priority_label. The table contains 5 rows of data.

issue_type	priority	priority_label
Login Issue	1	Low
Server down	3	High
Slow system	2	Medium
Email error	2	Medium
Password reset	1	Low

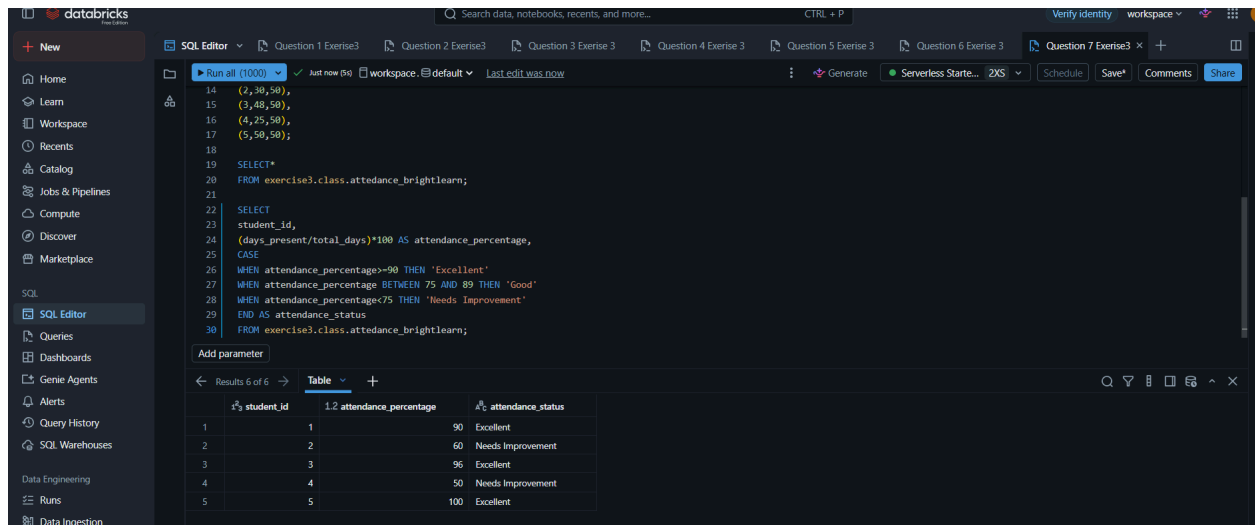
7.The Attendance Table

Calculate each student's attendance percentage and classify the result



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor (selected), Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, and Visual Data Prep. The main editor area displays SQL code for creating a catalog, schema, and table, followed by an insert statement and a query to calculate attendance percentages and status.

```
1 -- Step 1: CREATE A Catalog
2 CREATE CATALOG IF NOT EXISTS exercise3;
3
4 CREATE SCHEMA IF NOT EXISTS exercise3.class;
5
6 CREATE TABLE IF NOT EXISTS exercise3.class.attendance_brightlearn (
7   student_id INT,
8   days_present INT,
9   total_days INT
10 );
11
12 INSERT INTO TABLE exercise3.class.attendance_brightlearn VALUES
13 (1,45,50),
14 (2,30,50),
15 (3,40,50),
16 (4,25,50),
17 (5,50,50);
18
19 SELECT*
20 FROM exercise3.class.attendance_brightlearn;
21
22 SELECT
23   student_id,
24   (days_present/total_days)*100 AS attendance_percentage,
25   CASE
26     WHEN attendance_percentage>90 THEN 'Excellent'
27     WHEN attendance_percentage BETWEEN 75 AND 89 THEN 'Good'
28     WHEN attendance_percentage<75 THEN 'Needs Improvement'
29   END AS attendance_status
30 FROM exercise3.class.attendance_brightlearn;
```

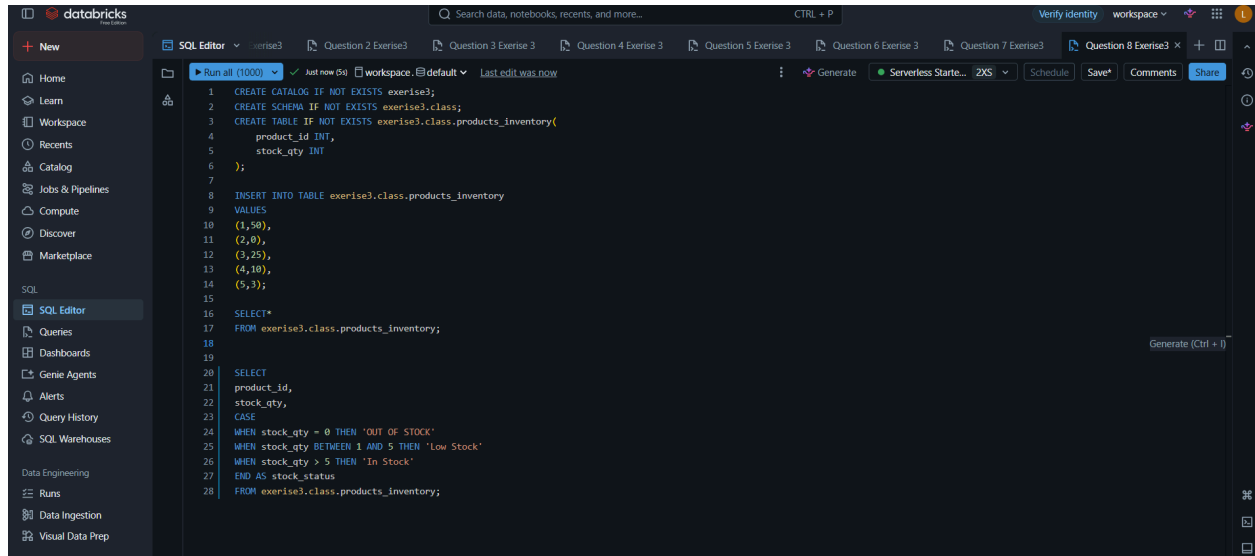


The screenshot shows the Databricks SQL Editor interface with the results of the query displayed in a table format. The table has three columns: student_id, attendance_percentage, and attendance_status. The results show five rows of data.

student_id	attendance_percentage	attendance_status
1	90	Excellent
2	60	Needs Improvement
3	96	Excellent
4	50	Needs Improvement
5	100	Excellent

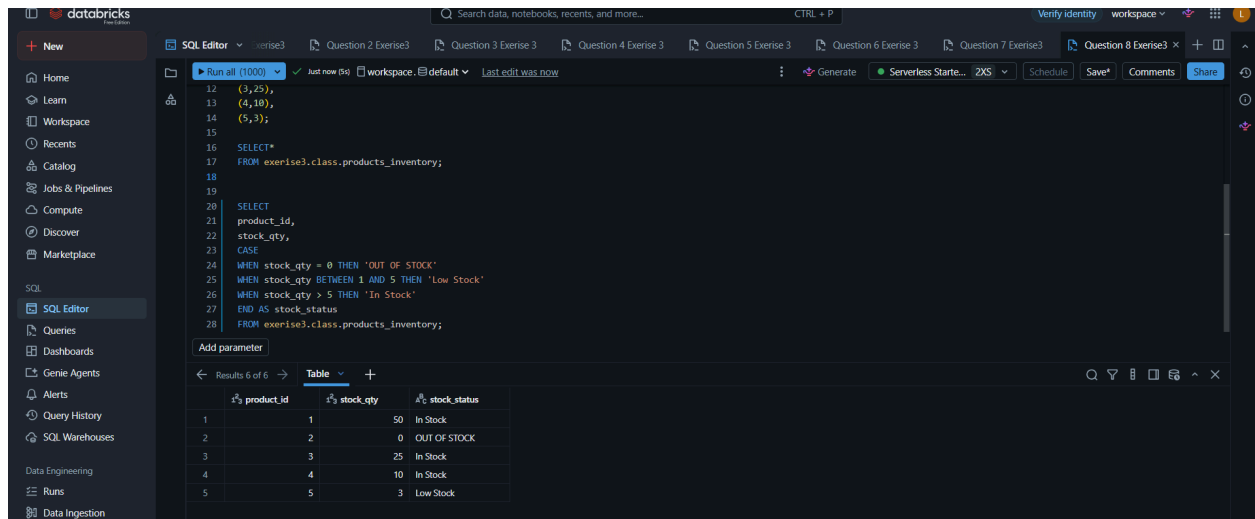
8. The products_inventory Table

Stock levels per product. Label each row with the correct stock status.



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays SQL code for creating a catalog, schema, and table, inserting data, and querying the table to determine stock status.

```
1 CREATE CATALOG IF NOT EXISTS exercise3;
2 CREATE SCHEMA IF NOT EXISTS exercise3.class;
3 CREATE TABLE IF NOT EXISTS exercise3.class.products_inventory(
4   product_id INT,
5   stock_qty INT
6 );
7
8 INSERT INTO TABLE exercise3.class.products_inventory
9 VALUES
10 (1,50),
11 (2,0),
12 (3,25),
13 (4,10),
14 (5,3);
15
16 SELECT*
17 FROM exercise3.class.products_inventory;
18
19
20 SELECT
21   product_id,
22   stock_qty,
23   CASE
24     WHEN stock_qty = 0 THEN 'OUT OF STOCK'
25     WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low Stock'
26     WHEN stock_qty > 5 THEN 'In Stock'
27   END AS stock_status
28 FROM exercise3.class.products_inventory;
```

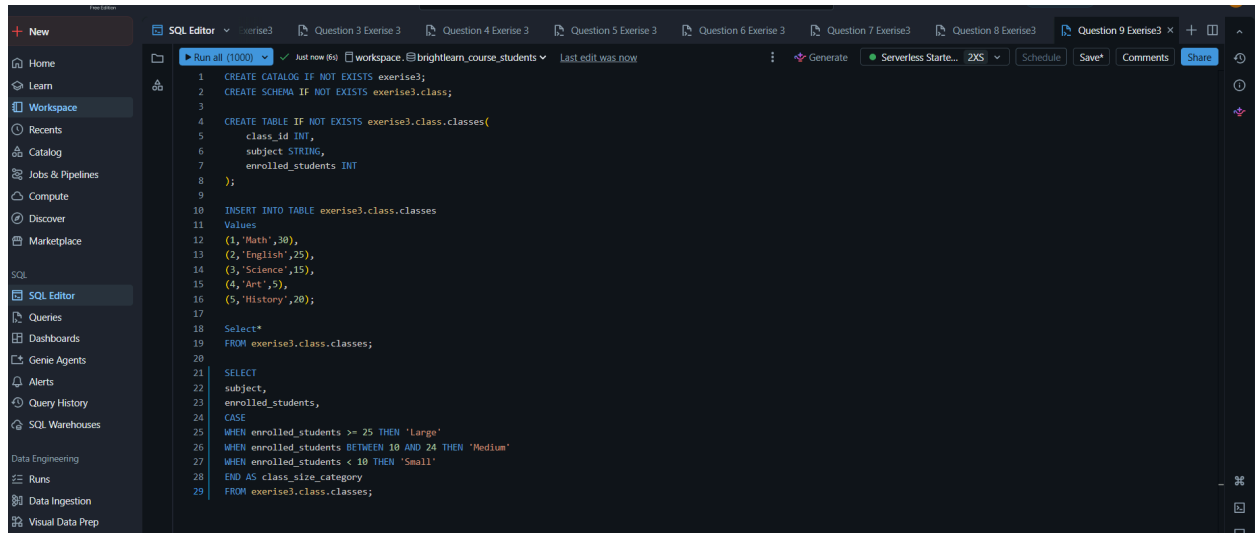


The screenshot shows the Databricks SQL Editor interface with the results of the SQL query displayed in a table. The table has three columns: product_id, stock_qty, and stock_status. The results show five rows of data.

product_id	stock_qty	stock_status
1	50	In Stock
2	0	OUT OF STOCK
3	25	In Stock
4	10	In Stock
5	3	Low Stock

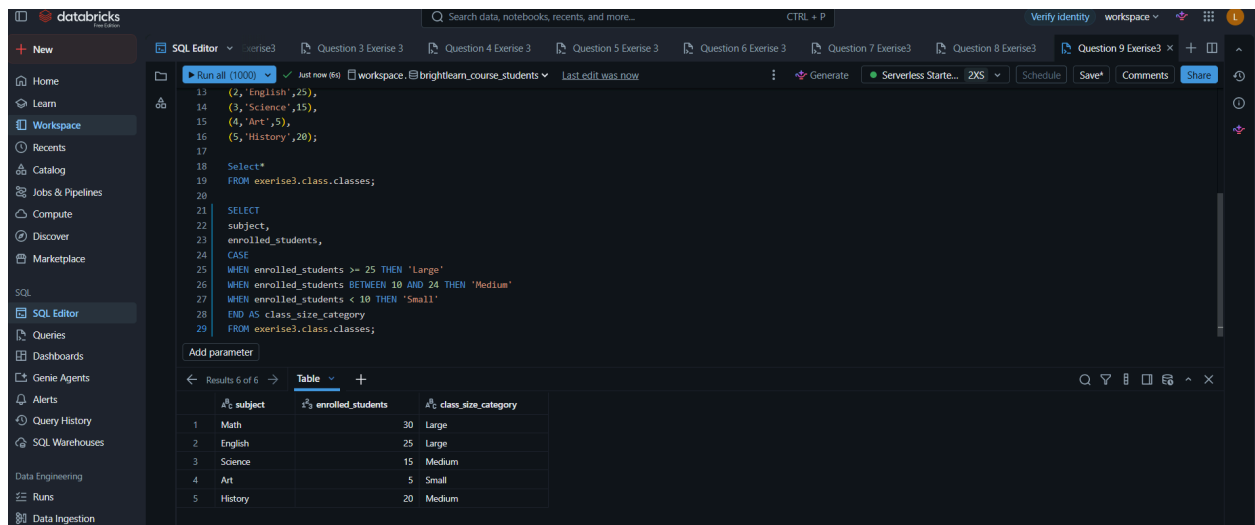
9. The classes Table

Classes with their enrolled student counts. Classify each class by size.



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area displays SQL code for creating a catalog, schema, and table, followed by an insert statement and a query to classify classes by size.

```
1 CREATE CATALOG IF NOT EXISTS exercise3;
2 CREATE SCHEMA IF NOT EXISTS exercise3.class;
3
4 CREATE TABLE IF NOT EXISTS exercise3.class.classes (
5   class_id INT,
6   subject STRING,
7   enrolled_students INT
8 );
9
10 INSERT INTO TABLE exercise3.class.classes
11 Values
12 (1, 'Math', 30),
13 (2, 'English', 25),
14 (3, 'Science', 15),
15 (4, 'Art', 5),
16 (5, 'History', 20);
17
18 Select*
19 FROM exercise3.class.classes;
20
21 SELECT
22   subject,
23   enrolled_students,
24   CASE
25     WHEN enrolled_students >= 25 THEN 'Large'
26     WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
27     WHEN enrolled_students < 10 THEN 'Small'
28   END AS class_size_category
29 FROM exercise3.class.classes;
```

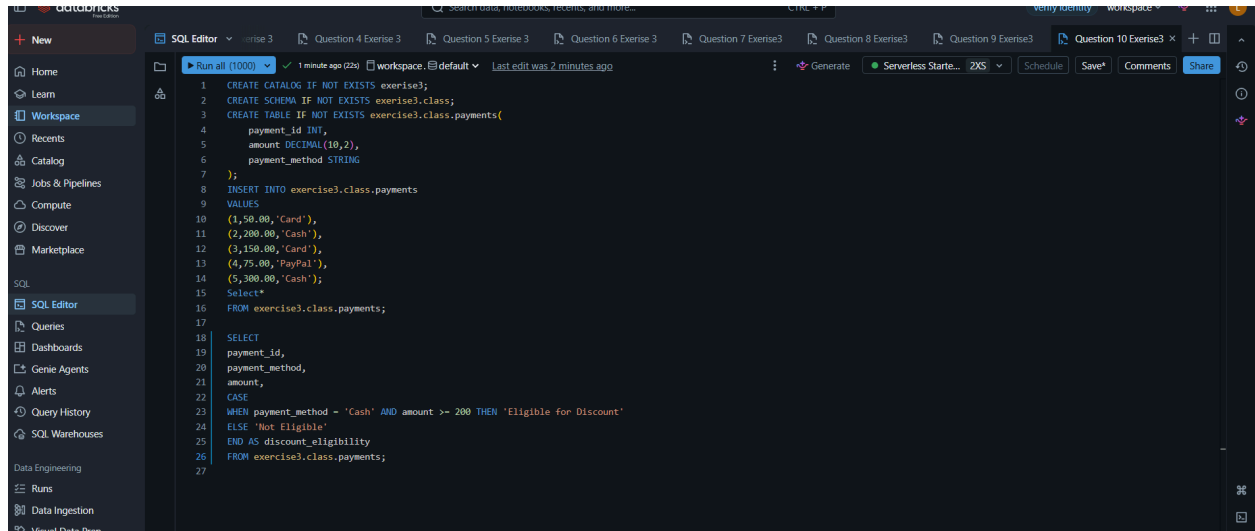


The screenshot shows the Databricks SQL Editor interface with the query results displayed below the code. The results are shown in a table format with columns: subject, enrolled_students, and class_size_category. The table contains 5 rows of data.

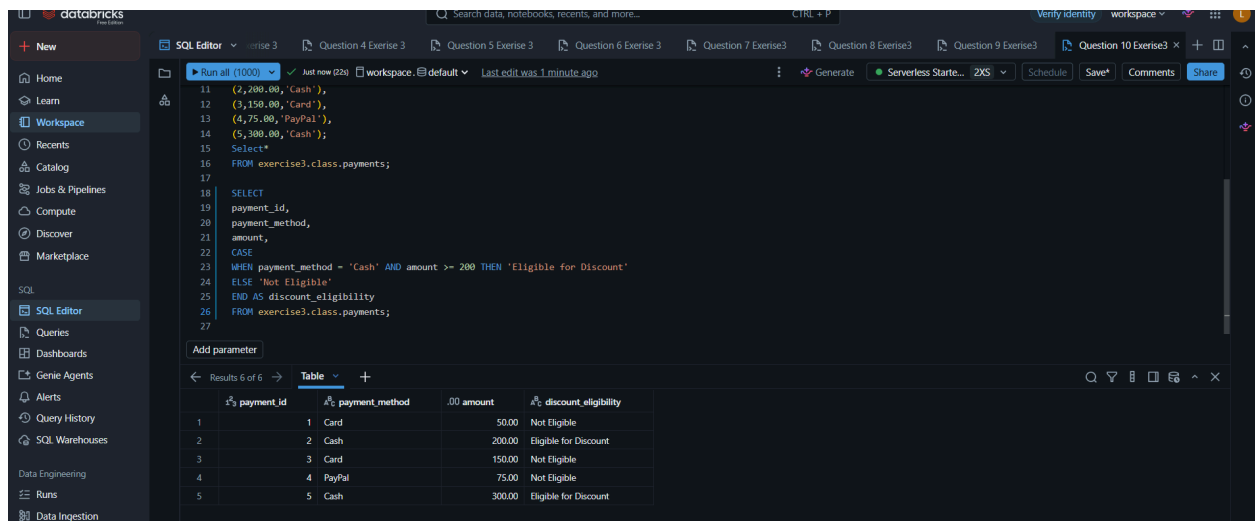
subject	enrolled_students	class_size_category
Math	30	Large
English	25	Large
Science	15	Medium
Art	5	Small
History	20	Medium

10. The paymentsTable

Combine the payment method and amount to decide whether a discount applies.



```
1 CREATE CATALOG IF NOT EXISTS exercise3;
2 CREATE SCHEMA IF NOT EXISTS exercise3.class;
3 CREATE TABLE IF NOT EXISTS exercise3.class.payments(
4     payment_id INT,
5     amount DECIMAL(10,2),
6     payment_method STRING
7 );
8 INSERT INTO exercise3.class.payments
9 VALUES
10 (1,50.00,'Card'),
11 (2,200.00,'Cash'),
12 (3,150.00,'Card'),
13 (4,75.00,'PayPal'),
14 (5,300.00,'Cash');
15 Select*
16 FROM exercise3.class.payments;
17
18 SELECT
19     payment_id,
20     payment_method,
21     amount,
22     CASE
23     WHEN payment_method = 'Cash' AND amount >= 200 THEN 'Eligible for Discount'
24     ELSE 'Not Eligible'
25     END AS discount_eligibility
26 FROM exercise3.class.payments;
27
```



	payment_id	payment_method	amount	discount_eligibility
1	1	Card	50.00	Not Eligible
2	2	Cash	200.00	Eligible for Discount
3	3	Card	150.00	Not Eligible
4	4	PayPal	75.00	Not Eligible
5	5	Cash	300.00	Eligible for Discount